

Documentación de Arquitectura

Milpa: Milpa

Documentación de Arquitectura del Sistema

Contents

Documentación de Arquitectura	1
1. Introducción	3
1.1 Propósito	3
1.2 Objetivo del Sistema	3
1.3 Stack Tecnológico	3
2. Arquitectura del Sistema	3
2.1 Diagrama de Alto Nivel	3
2.2 Flujo de Comunicación	4
3. Arquitectura del Backend	4
3.1 Diagrama Hexagonal	4
3.2 Capas del Sistema	4
3.3 Entidades de Dominio	5
4. Casos de Uso	5
4.1 Módulo de Usuarios	5
4.2 Módulo de Marketplace	5
4.3 Módulo de Chat	5
4.4 Módulo de Supply Chain	5
4.5 Módulo de Moderación	5
5. API REST	5
5.1 Endpoints	5
5.2 WebSocket	6
6. Base de Datos	6
6.1 Diagrama DER	6
6.2 Migraciones	6
6.3 Tablas por Módulo	6
7. Seguridad	6
7.1 Autenticación	6
7.2 Autorización	6
7.3 Middleware	7
8. Patrones y Estilos Arquitectónicos	7
8.1 Patrones de Diseño (GoF)	7
Builder	7
Decorator	7
Chain of Responsibility	7
State Machine	7
8.2 Patrones de Arquitectura	7
8.3 Patrones de Acceso a Datos (PoEAA)	7
9. Estructura del Código	8
10. Decisiones Técnicas	8
10.1 Go	8
10.2 PostgreSQL	8
10.3 Elasticsearch	8

10.4 Redis	8
11. Diagramas de base de datos	9
11.1 Diagrama Entidad Relación	10
Esquema físic	10

1. Introducción

1.1 Propósito

Este documento describe la arquitectura del sistema Milpa, una plataforma digital de mercado agropecuario que conecta directamente a pequeños y medianos productores de frutas y cítricos con compradores minoristas y mayoristas en Nicaragua.

1.2 Objetivo del Sistema

Eliminar la cadena de intermediación que encarece los productos y reduce el beneficio del productor, permitiendo un canal directo de venta que disminuya el desperdicio de cosechas y ofrezca precios más justos para ambas partes.

1.3 Stack Tecnológico

- **Frontend:** React 19 + Vite 8
- **Backend:** Go 1.26
- **Base de Datos:** PostgreSQL 16
- **Caché:** Redis 7
- **Búsqueda:** Elasticsearch 8
- **Chat:** WebSocket (gorilla/websocket)
- **Auth:** JWT (HS256) + bcrypt

2. Arquitectura del Sistema

2.1 Diagrama de Alto Nivel

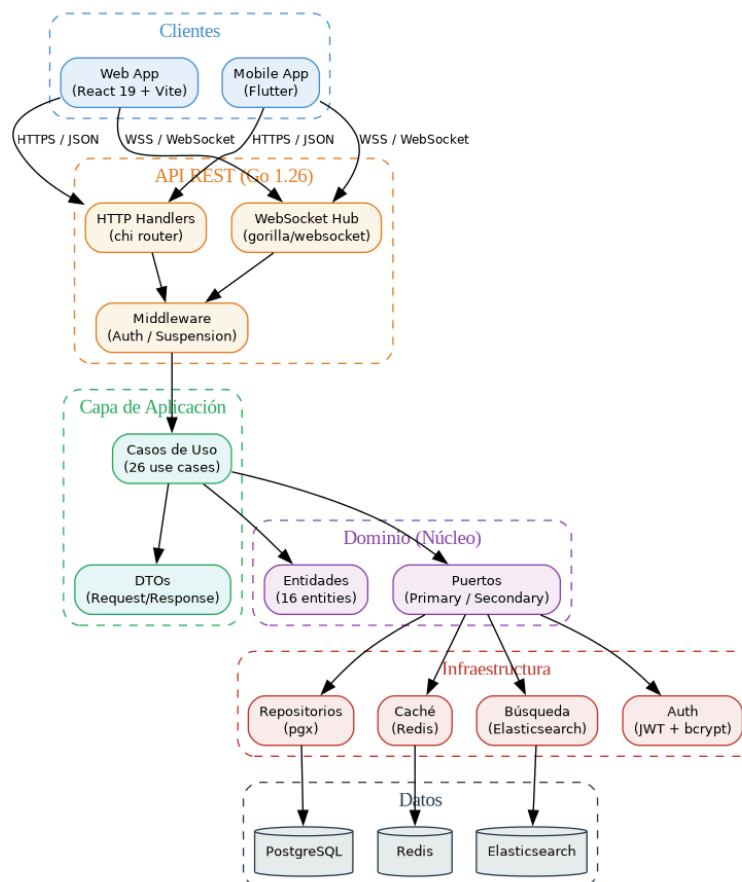


Figura 1: Diagrama de arquitectura del sistema.

2.2 Flujo de Comunicación

- **API REST:** JSON sobre HTTPS, autenticación mediante JWT (Bearer token).
- **Chat:** WebSocket sobre WSS, autenticación mediante subprotocolo bearer.<token>.
- **Búsqueda:** Elasticsearch con indexación automática al crear o actualizar ofertas.
- **Caché:** Redis con claves scoped por viewer para protección de datos personales.

3. Arquitectura del Backend

El backend implementa **Arquitectura Hexagonal** (Puertos y Adaptadores) para separar el dominio del núcleo de los detalles de infraestructura, permitiendo mantener la testabilidad y la evolución independiente de cada capa.

3.1 Diagrama Hexagonal

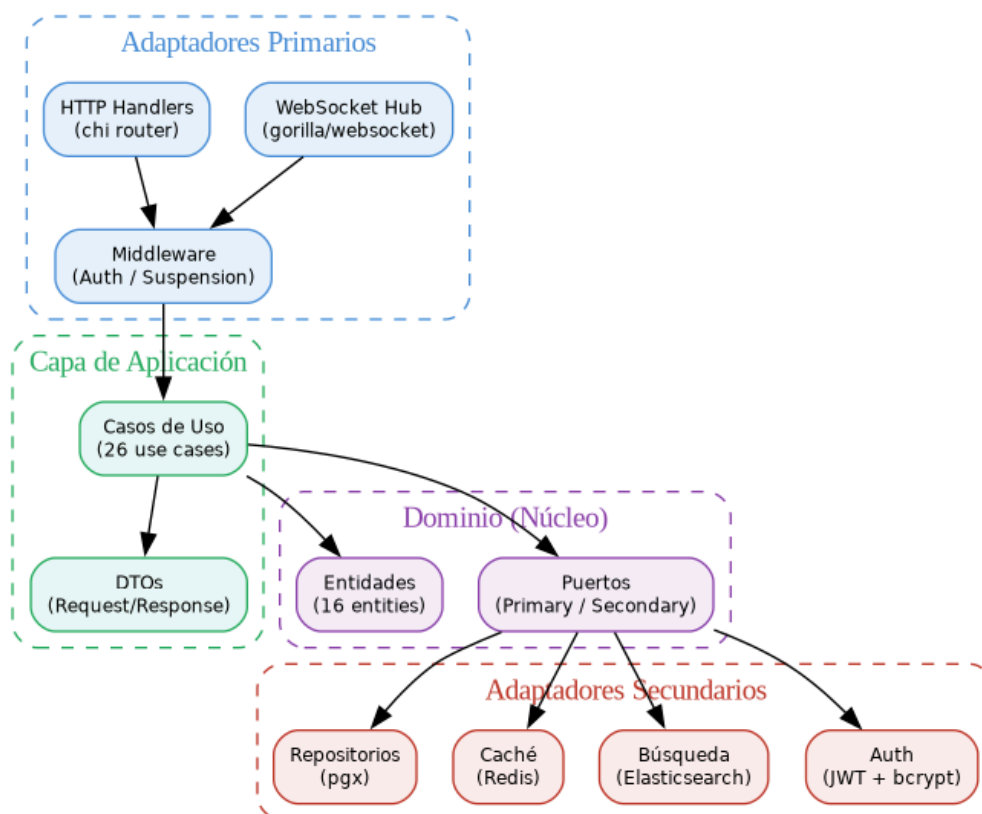


Figura 2: Arquitectura hexagonal del backend.

3.2 Capas del Sistema

Dominio (domain/entities) — Entidades, value objects y reglas de negocio. Sin dependencias externas.

Puertos Primarios (domain/port/primary) — Interfaces de casos de uso que el dominio expone.

Puertos Secundarios (domain/port/secondary) — Interfaces de repositorios, caché, JWT y otros servicios.

Aplicación (application/use-cases) — Casos de uso que orquestan la lógica de negocio.

DTOs (application/dto) — Objetos de transferencia de datos entre capas.

Adaptadores Primarios (infrastructure/adapters/primary/api) — Handlers HTTP, WebSocket y middleware.

Adaptadores Secundarios (infrastructure/adapters/secondary) — Implementaciones de repositorios, caché, auth y búsqueda.

3.3 Entidades de Dominio

Entidad	Responsabilidad
User, “Usuario del sistema con rol y autenticación”	Address, “Ubicación geográfica con coordenadas”
Company, “Empresa agrícola con verificación”	Category, “Clasificación de productos”
Offering, “Publicación de producto en el marketplace”	Inquiry, “Consulta de comprador a agricultor”
Review, “Calificación entre usuarios”	Liquidation, “Oferta de liquidación de lote”
Report, “Reporte de contenido fraudulento”	AuditLog, “Registro de operaciones de moderación”
Conversation, “Chat entre comprador y agricultor”	Message, “Mensaje individual del chat”
SupplyRequest, “Solicitud de abastecimiento mayorista”	SupplyOffer, “Oferta de agricultor a solicitud”
Match, “Aceptación de oferta (estilo Tinder)”	Transaction, “Ciclo de vida de la transacción”
SupplierInventory, “Inventario del proveedor”	

4. Casos de Uso

4.1 Módulo de Usuarios

Registro, autenticación, consulta y actualización de perfil.

4.2 Módulo de Marketplace

CRUD de ofertas, búsqueda con filtros y recomendación por cercanía, precio y reputación.

4.3 Módulo de Chat

Conversaciones, mensajería en tiempo real vía WebSocket.

4.4 Módulo de Supply Chain

Solicitudes de abastecimiento, ofertas de agricultores, match estilo Tinder y ciclo de transacción con confirmaciones.

4.5 Módulo de Moderación

Reportes, acciones de moderación (aprobar, rechazar, suspender) y registro de auditoría.

5. API REST

5.1 Endpoints

Método	Endpoint	Descripción
POST, /auth/register, “Registro de usuario”	POST, /auth/login, “Autenticación + JWT”	GET, /offerings, “Listar ofertas”
POST, /offerings, “Crear oferta”	GET, /search, “Búsqueda con filtros”	POST, /inquiries, “Crear consulta”

Método	Endpoint	Descripción
POST, /conversations, “Iniciar chat”	GET, /conversations/:id/messages, “Historial de mensajes”	POST, /supply-requests, “Publicar solicitud”
POST, /supply-offers, “Ofertar a solicitud”	POST, /matches/like/:offerID, “Aceptar oferta”	POST, /matches/pass/:offerID, “Rechazar oferta”
POST, /transactions/:id/confirm-start, “Confirmar inicio”	POST, /transactions/:id/confirm-delivery, “Confirmar entrega”	POST, /reports, “Reportar publicación”
PATCH, /admin/users/:id/suspend, “Suspende usuario”	GET, /admin/audit-logs, “Ver auditoría”	

5.2 WebSocket

- **Ruta:** GET /ws/:conversationID
- **Protocolo:** Subprotocolo milpa.chat.v1 + bearer.<token>
- **Mensajes:** JSON con id, conversation_id, sender_id, content, created_at

6. Base de Datos

6.1 Diagrama DER

Ver archivo docs/diagrams/milpa_db.puml (PlantUML).

6.2 Migraciones

19 migraciones SQL consolidadas en server/infrastructure/adapters/secondary/repository/migrations/.

6.3 Tablas por Módulo

- **Núcleo:** users, addresses, companies, categories, units_of_measure
- **Marketplace:** offerings, inquiries, reviews, liquidations
- **Supply Chain:** supply_requests, supply_offers, matches, transactions, supplier_inventory
- **Chat:** conversations, messages
- **Moderación:** reports, audit_logs

7. Seguridad

7.1 Autenticación

JWT HS256 con expiración de 24 horas. Claims: user_id, role. Contraseñas con bcrypt (cost 10).

7.2 Autorización

Rol	Permisos
Buyer, “Comprar, chatear, calificar, reportar”	Producer, “Publicar ofertas, gestionar catálogo, ofertar, chatear”
Admin, “Moderar, gestionar categorías, suspender usuarios, ver auditoría”	Auditor, “Solo lectura de información y auditoría”

7.3 Middleware

- **Authenticate** — Valida JWT, rechaza tokens expirados.
- **AuthenticateOptional** — Anota principal si existe, no rechaza.
- **CheckSuspension** — Bloquea usuarios suspendidos con 403.

8. Patrones y Estilos Arquitectónicos

8.1 Patrones de Diseño (GoF)

Builder

Creacional. Construcción de entidades con validación.

- `NewUser`, `NewOffering`, `NewMatch`, `NewTransaction`

Decorator

Estructural. Wrapper para agregar caché sin modificar el caso de uso.

- `CachedUserUseCase`, `CachedOfferingUseCase`, `CachedSearchUseCase`

Chain of Responsibility

Comportamiento. Middleware HTTP para procesamiento secuencial.

- `Authenticate` → `CheckSuspension` → `Handler`

State Machine

Comportamiento. Transiciones de estado con validación.

- `Match: Active` → `Cancelled`
- `Transaction: Matched` → `InProgress` → `Completed` / `Cancelled`

8.2 Patrones de Arquitectura

- **Hexagonal Architecture** — Puertos y adaptadores
- **Layered Architecture** — Dominio → Aplicación → Infraestructura

8.3 Patrones de Acceso a Datos (PoEAA)

- **Repository** — Acceso a datos abstracto
- **Unit of Work** — Transacciones atómicas multi-entidad
- **Data Transfer Object** — Transferencia entre capas

9. Estructura del Código

```
server/
├── cmd/
│   ├── api/           # Entry point HTTP
│   └── migrate/       # Entry point migraciones
├── domain/
│   ├── entities/      # 16 entidades + errors
│   └── port/
│       ├── primary/   # Interfaces de casos de uso
│       └── secondary/ # Interfaces de repos, cache, JWT
├── application/
│   ├── dto/           # Request/Response structs
│   └── use-cases/     # 26 casos de uso + cache decorators
├── infrastructure/
│   ├── adapters/
│   │   ├── primary/api/ # Handlers, router, middleware, WS
│   │   └── secondary/   # Repos, cache, auth, search, storage
│   ├── config/        # Carga de configuración
│   ├── database/      # pgxpool + migraciones
│   └── searchService/  # Cliente Elasticsearch
├── internal/
│   ├── auth/          # Principal, context, protocolos
│   └── validate/      # Validación de requests
└── tests/
    ├── unitary/       # ~35 archivos
    └── integration/   # ~28 archivos (testcontainers)
```

10. Decisiones Técnicas

10.1 Go

Concurrencia nativa (goroutines) para WebSocket. Tipado fuerte para dominio complejo. Performance para API REST de alto tráfico.

10.2 PostgreSQL

ACID para transacciones financieras. JSONB para metadata flexible. Extensiones pgcrypto y uuid-osp.

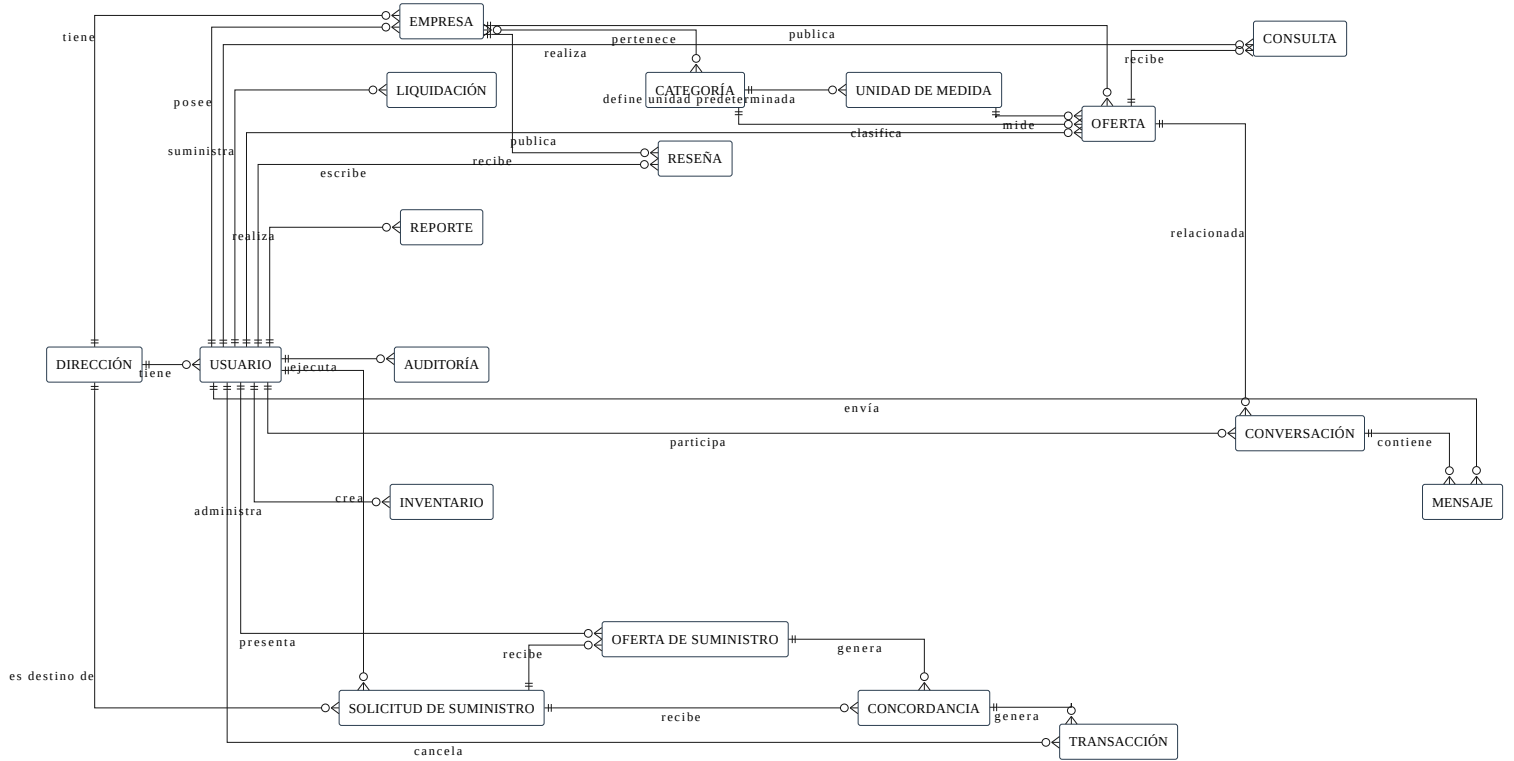
10.3 Elasticsearch

Búsqueda full-text con fuzziness. Geo-queries para ordenamiento por cercanía. Escalabilidad horizontal.

10.4 Redis

Cacheo de consultas frecuentes. Rate limiting futuro. Sesiones efímeras.

11. Diagramas de base de datos



11.1 Diagrama Entidad Relación

Esquema físic

